

NODE.JS CI/CD PIPELINE

Docker + GitHub Actions + AWS EC2

Project Documentation

Prepared by:

Sumit Milind Bhamkar

Technologies:

Node.js • Express.js • Docker • Docker Compose • Git • GitHub • GitHub Actions • AWS EC2
• Linux • SSH

1. Project Overview

This project demonstrates an end-to-end **CI/CD pipeline** for a Node.js and Express.js backend application.

The application is developed locally, containerized using Docker, stored in GitHub, and deployed on an AWS EC2 instance.

GitHub Actions is used to automate the deployment process whenever new code is pushed to the main branch.

The main objective of this project is to understand how application code moves from a developer's local environment to a running application on a cloud server through an automated CI/CD pipeline.

Project Objective

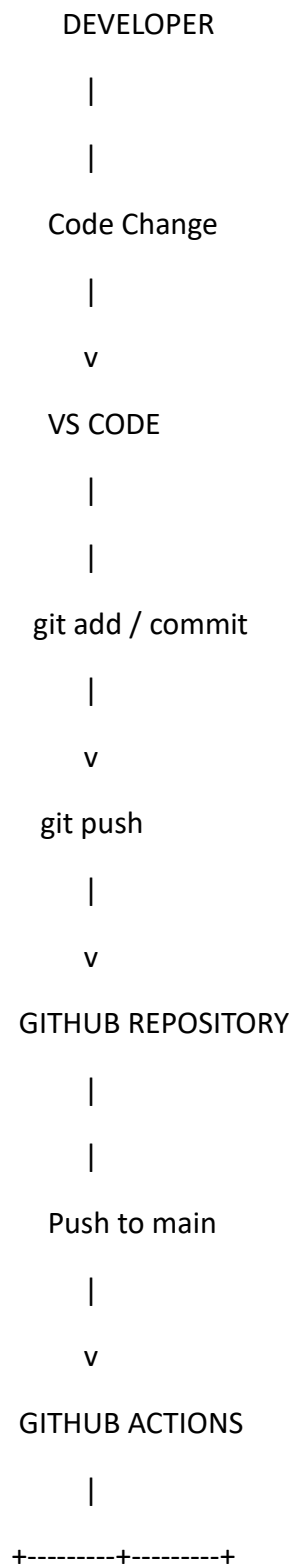
The project demonstrates:

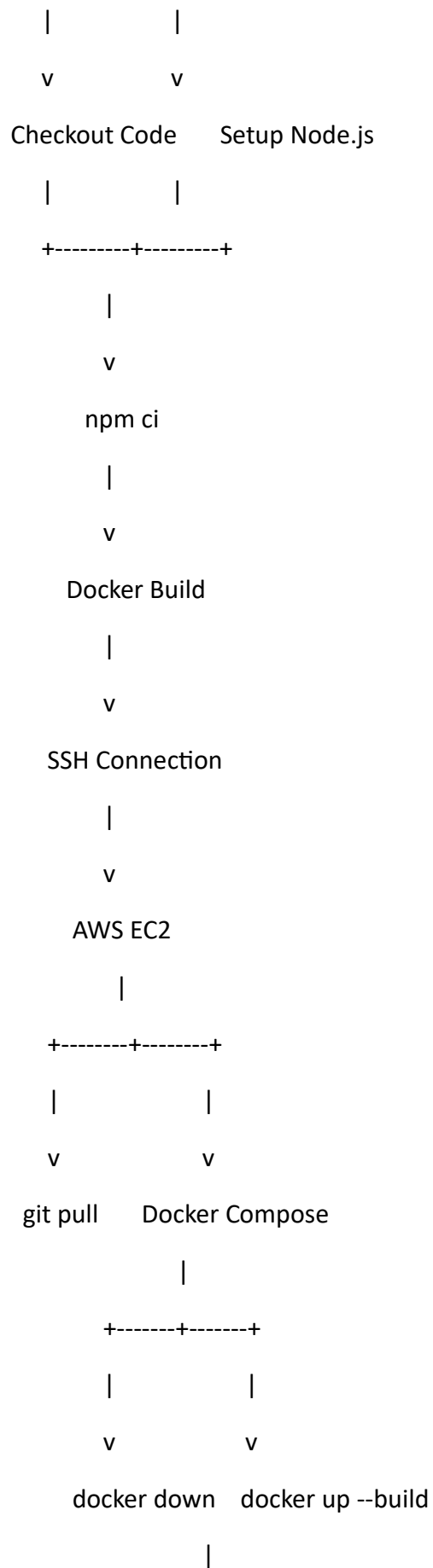
- Node.js backend development
- Docker containerization
- Docker Compose
- Git and GitHub
- AWS EC2 deployment
- Linux server management
- SSH authentication
- GitHub Actions
- Automated CI/CD

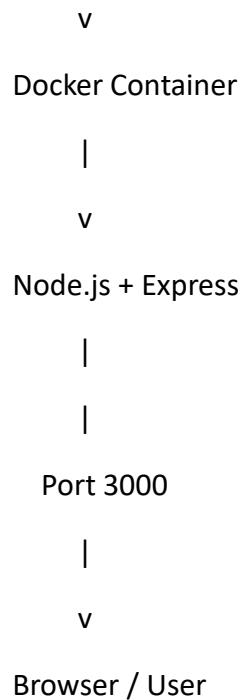
- Automated application deployment

2. Complete Project Flow

The complete flow implemented in this project is:







Flow Explanation

Step 1 — Developer

The developer makes a change to the Node.js application.

Step 2 — Git

The changes are added, committed, and pushed to the GitHub repository.

Step 3 — GitHub

The code is stored in the main branch.

Step 4 — GitHub Actions

The push triggers the GitHub Actions workflow automatically.

The workflow:

1. Checks out the latest source code.
2. Sets up Node.js.
3. Installs dependencies using npm ci.
4. Builds the Docker image.
5. Connects to the EC2 server using SSH.

Step 5 — AWS EC2

The EC2 server pulls the latest code from GitHub.

Step 6 — Docker Compose

Docker Compose stops the existing container and starts a new container using the updated code.

Step 7 — Application

The Node.js application starts inside the Docker container.

Step 8 — User

The updated application can be accessed through the EC2 public IP on port 3000.

In One Line

Code Change → Git Push → GitHub Actions → CI → Docker Build → SSH → EC2 → Git Pull → Docker Compose → Updated Application

3. Technologies Used

Technology	Purpose
Node.js	JavaScript runtime for the backend
Express.js	Framework used to build the API
Docker	Containerizes the application
Docker Compose	Manages the application container
Git	Version control
GitHub	Source code repository
GitHub Actions	CI/CD automation
AWS EC2	Cloud server for deployment
Linux	Operating system used on EC2
SSH	Secure connection for automated deployment

4. Project Structure

The project has the following structure:

node-backend-cicd/

```
|
|└─ .github/
|  └─ workflows/
|     └─ deploy.yml
|
|└─ src/
|   └─ server.js
|
|└─ .gitignore
|└─ Dockerfile
|└─ docker-compose.yml
|└─ package.json
└─ package-lock.json
```

Important Files

src/server.js

Contains the Node.js and Express application.

package.json

Contains project information, dependencies, and scripts.

Dockerfile

Contains instructions for creating the Docker image.

docker-compose.yml

Defines how the Docker container is built and executed.

.github/workflows/deploy.yml

Contains the GitHub Actions CI/CD pipeline.

.gitignore

Prevents unnecessary or sensitive files from being committed.

5. Node.js Application

The backend application was created using Node.js and Express.js.

The application listens on port 3000.

It contains two endpoints.

Root Endpoint

GET /

This endpoint returns the application status and message.

Example response:

```
{  
  "message": "Hello from DevOps CI/CD Pipeline",  
  "status": "running"  
}
```

Health Endpoint

GET /health

Example response:

```
{  
  "status": "healthy"  
}
```

Application Code

```
const express = require("express");  
  
const app = express();  
  
const PORT = 3000;  
  
app.get("/", (req, res) => {  
  res.json({  
    message: "Hello from DevOps CI/CD Pipeline",  
    status: "running"  
  });  
});
```

```
app.get("/health", (req, res) => {  
  res.json({  
    status: "healthy"  
  });  
});  
  
app.listen(PORT, () => {  
  console.log(`Server running on port ${PORT}`);  
});
```

6. Running the Application Locally

The Node.js project was initialized using:

```
npm init -y
```

Express was installed using:

```
npm install express
```

The application was started using:

```
npm start
```

The application was tested locally using:

```
http://localhost:3000
```

Health endpoint:

```
http://localhost:3000/health
```

7. Dockerization

The Node.js application was containerized using Docker.

Docker allows the application and its runtime environment to be packaged into a portable container.

Dockerfile

```
FROM node:22-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["npm", "start"]
```

Dockerfile Explanation

FROM

```
FROM node:22-alpine
```

Uses Node.js 22 Alpine as the base image.

WORKDIR

```
WORKDIR /app
```

Sets /app as the working directory inside the container.

COPY

```
COPY package*.json ./
```

Copies the package files into the container.

RUN

```
RUN npm install
```

Installs the Node.js dependencies.

COPY

```
COPY . .
```

Copies the application source code into the container.

EXPOSE

EXPOSE 3000

Documents that the application uses port 3000.

CMD

CMD ["npm", "start"]

Starts the Node.js application.

Docker Image Build

docker build -t node-backend-cicd:1.0 .

Run Docker Container

docker run -d -p 3000:3000 --name node-backend node-backend-cicd:1.0

8. Docker Compose

Docker Compose was used to simplify container management.

docker-compose.yml

services:

 backend:

 build: .

 container_name: node-backend

 ports:

 - "3000:3000"

Docker Compose allows the application to be started using:

docker compose up -d

To check the running container:

docker compose ps

To stop the application:

docker compose down

During deployment, Docker Compose is used with:

```
docker compose up -d --build
```

This rebuilds the Docker image using the latest source code.

9. Git and GitHub

Git was used for version control.

The project was initialized using:

```
git init
```

Files were added:

```
git add .
```

The initial commit was created:

```
git commit -m "Initial Node.js backend with Docker"
```

The project was connected to GitHub and pushed to the main branch.

Normal Development Flow

Code Change

```
|
```

```
v
```

```
git add .
```

```
|
```

```
v
```

```
git commit
```

```
|
```

```
v
```

```
git push
```

```
|
```

```
v
```

GitHub

The GitHub repository contains the complete project source code and CI/CD workflow.

10. AWS EC2 Deployment

An AWS EC2 instance was created to host the application.

The EC2 instance uses:

- Ubuntu 26.04 LTS
- Public IP address
- Docker
- Git
- Docker Compose

The EC2 instance acts as the deployment server.

Security Group

The following ports were configured for the learning project:

Port	Protocol	Purpose
22	TCP	SSH / server access
3000	TCP	Node.js application

Port 22 was restricted for manual access.

Port 3000 was opened for application testing.

For a real production deployment, application access should normally be protected using appropriate networking controls, HTTPS, a reverse proxy, or a load balancer.

11. Installing Docker on EC2

Docker was installed on the Ubuntu EC2 server.

The official Docker repository was configured because the initial Ubuntu package did not provide the expected Docker Compose command/plugin.

The Docker installation included:

- Docker Engine
- Docker CLI
- containerd
- Docker Buildx

- Docker Compose plugin

After installation, Docker was verified using:

```
docker --version
```

Docker Compose was verified using:

```
docker compose version
```

Git Installation

Git was installed using:

```
sudo apt install -y git
```

The project repository was then cloned:

```
git clone <repository-url>
```

The project directory was opened:

```
cd node-backend-cicd
```

12. Manual Deployment on EC2

Before implementing CI/CD, the application was manually deployed to EC2.

Inside the project directory:

```
sudo docker compose up -d --build
```

The running container was checked using:

```
sudo docker compose ps
```

The application was also tested directly from the EC2 server:

```
curl http://localhost:3000
```

Health endpoint:

```
curl http://localhost:3000/health
```

The application was then accessed through the EC2 public IP:

```
http://<EC2-PUBLIC-IP>:3000
```

This confirmed that the Dockerized Node.js application was successfully running on AWS EC2 before automation was added.

13. GitHub Actions CI/CD

GitHub Actions was used to automate the deployment process.

The workflow is triggered whenever code is pushed to the main branch.

Workflow

name: Node Backend CI/CD

on:

push:

branches:

- main

jobs:

deploy:

runs-on: ubuntu-latest

steps:

- name: Checkout code

uses: actions/checkout@v4

- name: Setup Node.js

uses: actions/setup-node@v4

with:

node-version: 22

- name: Install dependencies

run: npm ci

- name: Build Docker image

```
run: docker build -t node-backend-cicd:ci .
```

```
- name: Deploy to EC2
```

```
uses: appleboy/ssh-action@v1
```

```
with:
```

```
host: ${ secrets.EC2_HOST }
```

```
username: ${ secrets.EC2_USER }
```

```
key: ${ secrets.EC2_SSH_KEY }
```

```
script: |
```

```
cd ~/node-backend-cicd
```

```
git pull origin main
```

```
sudo docker compose down
```

```
sudo docker compose up -d --build
```

```
sudo docker compose ps
```

14. GitHub Actions Workflow Explanation

1. Trigger

```
on:
```

```
push:
```

```
branches:
```

```
- main
```

The workflow starts whenever code is pushed to the main branch.

2. Checkout

```
actions/checkout@v4
```

Downloads the latest repository code into the GitHub Actions runner.

3. Node.js Setup

Node.js 22 is configured for the CI environment.

4. Install Dependencies

```
npm ci
```

Installs dependencies from package-lock.json.

5. Docker Build

```
docker build -t node-backend-cicd:ci .
```

Builds the Docker image to validate that the Dockerfile can successfully build.

6. SSH Deployment

GitHub Actions connects to the EC2 server using SSH.

7. Pull Latest Code

```
git pull origin main
```

The EC2 server downloads the latest code.

8. Stop Existing Container

```
sudo docker compose down
```

Stops the existing deployment.

9. Rebuild and Start

```
sudo docker compose up -d --build
```

Builds the latest Docker image and starts the updated application.

10. Verify

```
sudo docker compose ps
```

Checks the container status.

15. GitHub Secrets

The following secrets were configured in the GitHub repository:

EC2_HOST

EC2_USER

EC2_SSH_KEY

EC2_HOST

Contains the public IP address of the EC2 instance.

EC2_USER

Contains the EC2 username:

ubuntu

EC2_SSH_KEY

Contains the private SSH key used by GitHub Actions to connect to EC2.

The private key was stored only in GitHub Secrets and was not committed to the repository.

Security Note

Private keys, AWS credentials, passwords, and other sensitive information should never be committed to a public GitHub repository.

16. CI/CD Demonstration

The pipeline was tested using a real application change.

Initially, the application returned:

```
{  
  "message": "Hello from DevOps CI/CD Pipeline",  
  "status": "running"  
}
```

The server.js file was then modified.

For example, the message was changed to:

message: "Hello from my automated CI/CD pipeline"

The change was committed:

```
git add .  
git commit -m "Update backend message"
```

The change was pushed:

```
git push
```

What happened after the push?

Developer changes server.js

|

v

git push

|

v

GitHub Repository

|

v

GitHub Actions starts

|

v

CI Process

|

v

Docker Build

|

v

SSH to EC2

|

v

git pull origin main

|

v

Docker Compose rebuild

|

v

Updated Container

|

v

Updated Application

After the GitHub Actions workflow completed successfully, the application was refreshed in the browser.

The updated response was displayed.

This verified that the code change had successfully travelled through the complete CI/CD pipeline.

17. Testing and Verification

The application was tested at different stages.

Local Application

`http://localhost:3000`

Health Endpoint

`http://localhost:3000/health`

Docker

The application was tested inside a Docker container.

Docker Compose

The application was tested using Docker Compose.

EC2

The application was deployed successfully on AWS EC2.

Browser

The application was accessed using:

`http://<EC2-PUBLIC-IP>:3000`

CI/CD

A code change was pushed to GitHub and automatically deployed to EC2.

The updated application response was verified after deployment.

18. Troubleshooting

Docker Compose Issue on Ubuntu 26.04

During the initial EC2 setup, the Ubuntu Docker package was installed.

However, the following command was not available as expected:

docker compose

The Docker Compose plugin package was also unavailable through the initial package source.

Solution

The initial Docker packages were removed.

Docker was then installed using Docker's official Ubuntu repository.

The installation included:

docker-ce

docker-ce-cli

containerd.io

docker-buildx-plugin

docker-compose-plugin

After installation, Docker Compose was successfully available.

This troubleshooting experience demonstrated the importance of checking Linux distribution versions and package compatibility when installing DevOps tools.

19. Security Considerations

Security was considered during the project setup.

SSH Key

The deployment private SSH key was stored in GitHub Secrets.

It was not committed to GitHub.

Git Ignore

The .gitignore file contains:

node_modules/

.env

This prevents unnecessary or potentially sensitive files from being committed.

Security Group

SSH access was restricted for manual access.

Application port 3000 was opened for learning and testing.

Production Consideration

This project is designed as a learning project.

For a production environment, additional security measures should be implemented, such as:

- HTTPS
 - Domain name
 - Nginx or another reverse proxy
 - Restricted network access
 - Load balancer
 - Proper IAM permissions
 - Centralized logging
 - Monitoring
 - Secret management
 - More controlled deployment strategies
-

20. Cleanup

After completing the project, AWS resources should be cleaned up to prevent unnecessary charges.

Stop the Docker Compose application if required:

```
sudo docker compose down
```

After completing the project, terminate the EC2 instance from the AWS EC2 console.

Also review the AWS account for unused:

- EC2 instances
- Elastic IPs
- EBS volumes
- Security groups
- Load balancers
- NAT gateways
- Other billable resources

The project was created as a learning environment, so unused AWS resources should be removed after testing.

21. Future Improvements

The current project uses a simple deployment model suitable for learning.

Possible improvements include:

Docker Registry

Instead of rebuilding the image directly on EC2, the pipeline could push the image to:

- Docker Hub
- Amazon ECR

EC2 could then pull a specific image version.

HTTPS

Nginx and SSL/TLS certificates could be added to provide secure HTTPS access.

Load Balancer

An AWS Application Load Balancer could be introduced to distribute traffic across multiple EC2 instances.

Automated Testing

Unit and integration tests could be added to the CI stage before deployment.

Monitoring

AWS monitoring and logging services could be integrated.

Zero-Downtime Deployment

The current deployment stops the existing container before starting the updated container.

A more advanced deployment strategy could reduce or eliminate application downtime.

Infrastructure as Code

The AWS infrastructure could be created using:

- Terraform
- AWS CloudFormation

This would make the infrastructure reproducible and version-controlled.

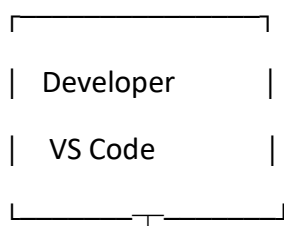
22. Key DevOps Concepts Learned

Through this project, the following concepts were practically implemented:

- Version Control using Git
- GitHub repositories
- Branches and commits
- Node.js application deployment
- Docker images
- Docker containers
- Dockerfile
- Docker Compose
- Linux server administration
- AWS EC2
- Security Groups
- SSH authentication
- GitHub Actions
- Continuous Integration
- Continuous Deployment
- GitHub Secrets
- Automated deployment
- Application testing
- Troubleshooting

23. Final Project Flow

The complete project can be summarized as:



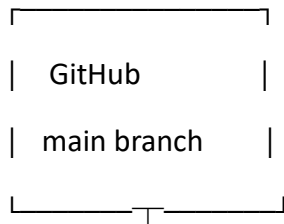
|
Code Change

|
v



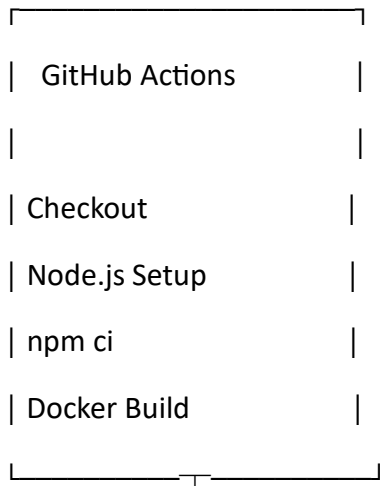
|
git push

|
v



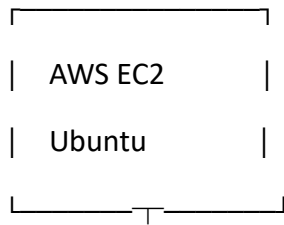
|
Trigger

|
v



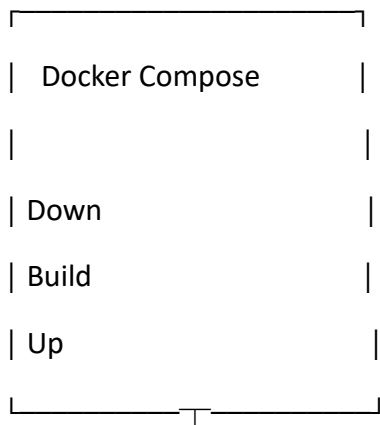
|
SSH Deploy

|
v

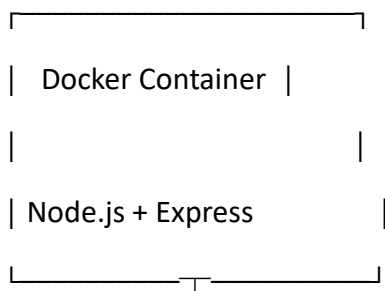


|
git pull

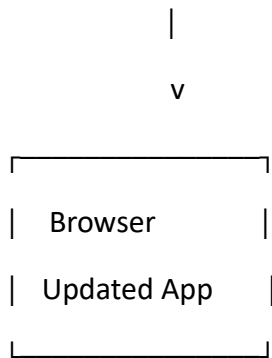
|
v



|
v



|
Port 3000



Final Summary

Code Change



Git Push



GitHub



GitHub Actions



CI Validation



Docker Build



SSH



AWS EC2



Git Pull



Docker Compose



Updated Container



Node.js Application



Browser

This demonstrates the complete journey of application code from development to automated cloud deployment.

24. Reproduce This Project

The complete source code and CI/CD configuration are available in the GitHub repository.

GitHub Repository

[Add your GitHub repository link here]

The repository contains:

Source Code

Dockerfile

Docker Compose

GitHub Actions Workflow

Package Configuration

.gitignore

README.md

Documentation

The complete project documentation is available in this PDF:

`docs/PROJECT-DOCUMENTATION.pdf`

Demo

A recorded demo demonstrates the CI/CD process:

Code Change



Git Push



GitHub Actions



Successful Deployment



Refresh Browser



Updated Application

[Add your demo video link here]

25. Conclusion

This project provided practical experience in building and deploying a Node.js application using modern DevOps tools and practices.

The project connected multiple technologies into a single automated workflow:

Git

+

GitHub

+

GitHub Actions

+

Docker

+

Docker Compose

+

Linux

+

AWS EC2

+

SSH

=

CI/CD Deployment Pipeline

The most important outcome of the project was understanding the complete flow of application deployment rather than using each technology independently.

A code change made by the developer can now be pushed to GitHub and automatically deployed to the Dockerized Node.js application running on AWS EC2.

This project forms a practical foundation for exploring more advanced DevOps concepts such as container registries, Amazon ECR, load balancers, HTTPS, monitoring, Infrastructure as Code, and zero-downtime deployments.

Project Information

Project: Node.js CI/CD Pipeline

Platform: AWS EC2

Application: Node.js + Express.js

Containerization: Docker + Docker Compose

CI/CD: GitHub Actions

Version Control: Git + GitHub

Operating System: Ubuntu 26.04 LTS

Application Port: 3000

Author: Sumit Milind Bhamkar